

1. 如果每秒收到數百至數千筆 **Tick**, 你會如何避免資料庫與前端被壓垮？

- 後端與資料庫：
 - **Event Bus** 緩衝: 導入 Kafka 或 Redis Streams 作為消息隊列進行 Backpressure(背壓)處理。
 - 寫入分離與微批處理 (**Micro-batching**): 高頻 Tick 僅於記憶體/Redis 更新快取, 不直接每筆寫入 DB; 寫入 DB 的 Audit Log 僅限「觸發通知的訊號」或採用每 1 秒批次 Bulk Insert。
- 前端渲染：
 - **Throttle / Debounce** 批次渲染: 前端使用 `requestAnimationFrame` 或 100ms 進行 Throttling, 限制 DOM 與 Chart.js 繪圖頻率, 避免每秒千次刷新導致瀏覽器 Freeze。

2. 現貨時間為 **10:31:20.100**、期貨為 **10:31:21.300**, 是否應計算價差？為什麼？

- 不應該計算(判定為 **Invalid / Time Skew** 過大)。
- 原因: 兩者時間戳落差達 1.2 秒 (1200ms)。在即時套利中, 1 秒的時間差代表你拿「歷史現貨對上最新期貨」, 這極可能是市場已消化完畢的「幻影價差 (Phantom Spread)」。若依此訊號下單, 會面臨極高的腿差風險 (Legging Risk)。

3. 若使用者把門檻設為 **4%**, 行情在 **3.99%~4.01%** 間高速震盪, 你如何設計狀態機？

- 設計帶有 **Hysteresis** (遲滯/震盪抑制) 與狀態冷卻的狀態機：
 1. 首次觸發: 超過 4.0% 觸發通知, 記錄時間戳與當前利潤 $SP_0\$$ 。
 2. **Cooldown** 冷卻期: 觸發後進入 N 秒 (如 60 秒) 冷卻期, 期間即使行情跌下 3.99% 又升回 4.01%, 一律抑制通知。
 3. **Re-trigger** 增幅門檻: 冷卻期過後, 需滿足淨套利率比上次發送時再增加一定幅度 (如 $SP_0 + 0.5\% = 4.5\%$) 才允許發送第二次通知。
 4. 狀態重置: 只有當行情大幅回落至安全區塊 (如 $< 3.5\%$) 且過冷卻期後, 狀態機才會重置回初始可發送狀態。

4. 券商 **API** 斷線 15 秒後重連, 你如何處理期間缺口與舊 **Tick**？

- 舊 **Tick** 丟棄 (**Drop Stale Ticks**): 重連後接收到的歷史 Tick (時間戳小於重連時間點) 直接丟棄, 不得納入即時價差計算。
- 系統狀態切換為 **RECONNECTING**: 斷線期間與重連初期, 前端明確標示「數據同步中」, 禁止發送任何套利通知。
- 快照同步 (**Snapshot Catch-up**): 重連後主動調用 REST API 請求最新的 Order Book Snapshot, 重新校準現貨與期貨當下 Bid/Ask 點位, 確認數據同步後才恢復監控機制。

5. 為何套利判斷應使用 **Bid / Ask**, 而不是 **Last Price**？

- **Last Price** 是「歷史成交價」, 代表過去發生的交易, 不保證當下有足夠流動性供你以該價格成交。
- **Bid / Ask** 是「當下可成交價」: 當策略為「買現貨、賣期貨」時, 實際能立刻買到現貨的價格是 Spot Ask (賣一價), 能賣出期貨的價格是 Future Bid (買一價)。以 Bid/Ask 計算才能反映真實扣除買賣價差 (Bid-Ask Spread) 後的「實質可成交價差」。

6. 正式上線後，除息資料抓取失敗時系統應如何降級？

- 採用優雅降級 (Graceful Degradation) 策略：
 1. 標註風險狀態 (UNVERIFIED): 系統將除息點數降級標示為「資料未更新/未核驗」，並在前端顯示風險警示。
 2. 切換為保守邏輯: 系統自動暫停將預估除息點數納入正價差套利計算，改以原始價差 (Raw Spread) 進行保守評估。
 3. 自動告警: 內部監控發送 Alert 通報維運人員手動修復爬蟲或 API 來源。

7. 如果未來客戶要求自動下單，你認為現有架構要新增哪些風控模組？

- 必須擴充 5 大風控模組：
 1. **Pre-trade Fat-finger & Limit Guard**: 單筆委託張數上限、價格偏離度 (防止錯單與胖手指)。
 2. **Global Capital & Position Limit**: 總部位上限、單一商品持倉上限、帳戶總保證金安全率檢測。
 3. **Legging Risk** 執行引擎: 兩腿下單策略 (先下流動性差的一腿，成交後立即以 FOK/IOC 下第二腿; 若第一腿失敗立即撤單或平倉)。
 4. **Circuit Breaker** (每日虧損熔斷器): 每日累積虧損達門檻 (Drawdown Limit) 時，自動鎖定帳戶並停止所有下單。
 5. **API & 硬體防護鎖**: 硬體 Key 驗證、IP 白名單綁定、下單 API Rate Limit 護欄。